



Permissions in the Java Development Kit (JDK)

A permission represents access to a system resource. In order for a resource access to be allowed for an applet (or an application running with a security manager), the corresponding permission must be explicitly granted to the code attempting the access.

A permission typically has a name (often referred to as a "target name") and, in some cases, a comma-separated list of one or more actions. For example, the following code creates a `FilePermission` object representing read access to the file named `abc` in the `/tmp` directory:

```
perm = new java.io.FilePermission("/tmp/abc", "read");
```

In this, the target name is `/tmp/abc` and the action string is `"read"`.

Important Note: The above statement creates a permission object. A permission object represents, but does not grant access to, a system resource. Permission objects are constructed and assigned ("granted") to code based on the policy in effect. When a permission object is assigned to some code, that code is granted the permission to access the system resource specified in the permission object, in the specified manner. A permission object may also be constructed by the current security manager when making access decisions. In this case, the (target) permission object is created based on the requested access, and checked against the permission objects granted to and held by the code making the request.

The policy for a Java application environment is represented by a Policy object. In the `"JavaPolicy"` Policy implementation, the policy can be specified within one or more policy configuration files. The policy file(s) specify what permissions are allowed for code from specified code sources. A sample policy file entry that grants code from the `/home/sysadmin` directory read access to the file `/tmp/abc` is

```
grant codeBase "file:/home/sysadmin/" {  
    permission java.io.FilePermission "/tmp/abc", "read";  
};
```

For information about policy file locations and granting permissions in policy files, see [Default Policy Implementation and Policy File Syntax](#). For information about using the **Policy Tool** to specify the permissions, see the Policy Tool documentation ([for Solaris, Linux, or Mac OS X](#)) ([for Windows](#)). Using the **Policy Tool** saves typing and eliminates the need for you to know the required syntax of policy files.

Technically, whenever a resource access is attempted, *all* code traversed by the execution thread up to that point must have permission for that resource access, unless some code on the thread has been marked as "privileged." See [API for Privileged Blocks](#) for more information about "privileged" code.

This document contains tables that describe the built-in JDK permission types and discuss the risks of granting each permission. It also contains tables showing the methods that require permissions to be in effect in order to be successful, and for each lists the required permission.

The following topics are covered:

- **Permission Descriptions and Risks**

- `java.awt.AWTPermission`
- `java.io.FilePermission`
- `java.io.SerializablePermission`
- `java.lang.management.ManagementPermission`
- `java.lang.reflect.ReflectPermission`
- `java.lang.RuntimePermission`
 - NIO-Related Targets
- `java.net.NetPermission`
- `java.net.SocketPermission`
- `java.net.URLPermission`
- `java.nio.file.LinkPermission`
- `java.security.AllPermission`
- `java.security.SecurityPermission`
- `java.security.UnresolvedPermission`
- `java.sql.SQLPermission`
- `java.util.logging.LoggingPermission`
- `java.util.PropertyPermission`
- `javax.management.MBeanPermission`
- `javax.management.MBeanServerPermission`
- `javax.management.MBeanTrustPermission`
- `javax.management.remote.SubjectDelegationPermission`
- `javax.net.ssl.SSLPermission`
- `javax.security.auth.AuthPermission`
- `javax.security.auth.kerberos.DelegationPermission`
- `javax.security.auth.kerberos.ServicePermission`
- `javax.security.auth.PrivateCredentialPermission`
- `javax.sound.sampled.AudioPermission`
- `javax.xml.bind.JAXBPermission`

- [javax.xml.ws.WebServicePermission](#)
- [Methods and the Permissions They Require](#)
- [java.lang.SecurityManager Method Permission Checks](#)

For more information about permissions, including the superclasses `java.security.Permission` and `java.security.BasicPermission`, and examples of creating permission objects and granting permissions, see the [Security Architecture Specification](#).

Permission Descriptions and Risks

The following tables describe the built-in JDK permission types and discuss the risks of granting each permission.

AWTPermission

A `java.awt.AWTPermission` is for AWT permissions.

The following table lists all the possible `AWTPermission` target names, and for each provides a description of what the permission allows and a discussion of the risks of granting code the permission.

java.awt.AWTPermission Target Name	What the Permission Allows	Risks of Allowing this Permission
<code>accessClipboard</code>	Posting and retrieval of information to and from the AWT clipboard	This would allow malfeasant code to share potentially sensitive or confidential information.
<code>accessEventQueue</code>	Access to the AWT event queue	After retrieving the AWT event queue, malicious code may peek at and even remove existing events from the system, as well as post bogus events which may purposefully cause the application or applet to misbehave in an insecure manner.
<code>accessSystemTray</code>	Access to the AWT SystemTray instance	This would allow malicious code to add tray icons to the system tray. First, such an icon may look like the icon of some known application (such as a firewall or anti-virus) and order a user to do something unsafe (with help of balloon messages). Second, the system tray may be glutted with tray icons so that no one could add a tray icon anymore.
<code>createRobot</code>	Create <code>java.awt.Robot</code> objects	The <code>java.awt.Robot</code> object allows code to generate native-level mouse and keyboard events as well as read the screen. It could allow malicious code to control the system, run other programs, read the display, and deny mouse and keyboard access to the user.
<code>fullScreenExclusive</code>	Enter full-screen exclusive mode	Entering full-screen exclusive mode allows direct access to low-level graphics card memory. This could be used to spoof the system, since the program is in direct control of rendering.
<code>listenToAllAWTEvents</code>	Listen to all AWT events, system-wide	After adding an AWT event listener, malicious code may scan all AWT events dispatched in the system, allowing it to read all user input (such as passwords). Each AWT event listener is called from

		within the context of that event queue's <code>EventDispatchThread</code> , so if the <code>accessEventQueue</code> permission is also enabled, malicious code could modify the contents of AWT event queues system-wide, causing the application or applet to misbehave in an insecure manner.
<code>readDisplayPixels</code>	Readback of pixels from the display screen	Interfaces such as the <code>java.awt.Composite</code> interface which allow arbitrary code to examine pixels on the display enable malicious code to snoop on the activities of the user.
<code>replaceKeyboardFocusManager</code>	Sets the <code>KeyboardFocusManager</code> for a particular thread.	When a <code>SecurityManager</code> is installed, the invoking thread must be granted this permission in order to replace the current <code>KeyboardFocusManager</code> . If permission is not granted, a <code>SecurityException</code> will be thrown.
<code>setAppletStub</code>	Setting the stub which implements Applet container services	Malicious code could set an applet's stub and result in unexpected behavior or denial of service to an applet.
<code>setWindowsAlwaysOnTop</code>	Setting always-on-top property of the window: <code>Window.setAlwaysOnTop(boolean)</code>	The malicious window might make itself look and behave like a real full desktop, so that information entered by the unsuspecting user is captured and subsequently misused.
<code>showWindowWithoutWarningBanner</code>	Display of a window without also displaying a banner warning that the window was created by an applet	Without this warning, an applet may pop up windows without the user knowing that they belong to an applet. Since users may make security-sensitive decisions based on whether or not the window belongs to an applet (entering a username and password into a dialog box, for example), disabling this warning banner may allow applets to trick the user into entering such information.
<code>toolkitModality</code>	Creating <code>TOOLKIT_MODAL</code> dialogs and setting the <code>TOOLKIT_EXCLUDE</code> window property.	When a toolkit-modal dialog is shown from an applet, it blocks all other applets in the browser. When launching applications from Java Web Start, its windows (such as the security dialog) may also be blocked by toolkit-modal dialogs, shown from these applications.
<code>watchMousePointer</code>	Getting the information about the mouse pointer position at any time	Constantly watching the mouse pointer, an applet can make guesses about what the user is doing, i.e. moving the mouse to the lower left corner of the screen most likely means that the user is about to launch an application. If a virtual keypad is used so that keyboard is emulated using the mouse, an applet may guess what is being typed.

FilePermission

A `java.io.FilePermission` represents access to a file or directory. A `FilePermission` consists of a pathname and a set of actions valid for that pathname.

Pathname is the pathname of the file or directory granted the specified actions. A pathname that ends in `/*` (where `/` is the file separator character, `File.separatorChar`) indicates a directory and all the files contained in that directory. A pathname that ends with `/-` indicates (recursively) all files and subdirectories contained in that directory. A pathname consisting of the special token `"<<ALL FILES>>"` matches **any** file.

A pathname consisting of a single `""` indicates all the files in the current directory, while a pathname consisting of a single `"-"` indicates all the files in the current directory and (recursively) all files and subdirectories contained in the current directory.

The actions to be granted are passed to the constructor in a string containing a list of zero or more comma-separated keywords. The possible keywords are `"read"`, `"write"`, `"execute"`, `"delete"`, and `"readLink"`. Their meanings are defined as follows:

Keyword	Meaning
read	Permission to read.
write	Permission to write (which includes permission to create).
execute	Permission to execute. Allows <code>Runtime.exec</code> to be called. Corresponds to <code>SecurityManager.checkExec</code> .
delete	Permission to delete. Allows <code>File.delete</code> to be called. Corresponds to <code>SecurityManager.checkDelete</code> .
readLink	Permission to read links. Allows the target of a symbolic link to be read by invoking the <code>readSymbolicLink</code> method.

The actions string is converted to lowercase before processing.

Be careful when granting `FilePermissions`. Think about the implications of granting read and especially write access to various files and directories. The "<<ALL FILES>>" permission with write action is especially dangerous. This grants permission to write to the entire file system. One thing this effectively allows is replacement of the system binary, including the JVM runtime environment.

Note: Code can always read a file from the same directory it is in (or a subdirectory of that directory); it does not need explicit permission to do so. Code can also obtain the pathname of the directory it is executed from, and this pathname may contain sensitive information. For example, if code is executed from a home directory (or a subdirectory of the home directory), the pathname may reveal the name of the current user.

SerializablePermission

A `java.io.SerializablePermission` is for serializable permissions. A `SerializablePermission` contains a name (also referred to as a "target name") but no actions list; you either have the named permission or you don't.

The target name is the name of the Serializable permission (see below).

The following table lists all the possible `SerializablePermission` target names, and for each provides a description of what the permission allows and a discussion of the risks of granting code the permission.

<code>java.io.SerializablePermission</code> Target Name	What the Permission Allows	Risks of Allowing this Permission
<code>enableSubclassImplementation</code>	Implementing a subclass of <code>ObjectOutputStream</code> or <code>ObjectInputStream</code> to override the default serialization or deserialization, respectively, of objects	Code can use this to serialize or deserialize classes in a purposefully malfeasant manner. For example, during serialization, malicious code can use this to purposefully store confidential private field data in a way easily accessible to attackers. Or, during deserializaiton it could, for example, deserialize a class with all its private fields zeroed out.
<code>enableSubstitution</code>	Substitution of one object for another during serialization or deserialization	This is dangerous because malicious code can replace the actual object with one which has incorrect or malignant data.

ManagementPermission

The permission which the `SecurityManager` will check when code that is running with a `SecurityManager` calls methods defined in the management interface for the Java platform.

The following table provides a summary description of what the permission allows, and discusses the risks of granting code the permission.

java.lang.management.ManagementPermission Target Name	What the Permission Allows	Risks of Allowing this Permission
control	Ability to control the runtime characteristics of the Java virtual machine, for example, enabling and disabling the verbose output for the class loading or memory system, setting the threshold of a memory pool, and enabling and disabling the thread contention monitoring support. Some actions controlled by this permission can disclose information about the running application, like the <code>-verbose:classpath</code> flag.	This allows an attacker to control the runtime characteristics of the Java virtual machine and cause the system to misbehave. An attacker can also access some information related to the running application.
monitor	Ability to retrieve runtime information about the Java virtual machine such as thread stack trace, a list of all loaded class names, and input arguments to the Java virtual machine.	This allows malicious code to monitor runtime information and uncover vulnerabilities.

Programmers do not normally create `ManagementPermission` objects directly. Instead they are created by the security policy code based on reading the security policy file.

ReflectPermission

A `java.lang.reflect.ReflectPermission` is for reflective operations. A `ReflectPermission` is a *named permission* and has no actions. The `suppressAccessChecks` name allows suppressing the standard language access checks -- for public, default (package) access, protected, and private members -- performed by reflected objects at their point of use. The `newProxyInPackage.{package name}` provide the ability to create a proxy instance in the specified package of which the non-public interface that the proxy class implements.

The following table provides a summary description of what the permission allows, and discusses the risks of granting code the permission.

java.lang.reflect.ReflectPermission Target Name	What the Permission Allows	Risks of Allowing this Permission
suppressAccessChecks	Warning: <i>Extreme caution should be taken before granting this permission to code</i> , for it provides the ability to access fields and invoke methods in a class. This includes not only public, but protected and private fields and methods as well.	This is dangerous in that information (possibly confidential) and methods normally unavailable would be accessible to malicious code.
newProxyInPackage.{package name}	Ability to create a proxy instance in the specified package of which the non-public interface that the proxy class implements.	This gives code access to classes in packages to which it normally does not have access and the dynamic proxy class is in the system protection domain. Malicious code may use these classes to help in its attempt to compromise security in the system.

RuntimePermission

A `java.lang.RuntimePermission` is for runtime permissions. A `RuntimePermission` contains a name (also referred to as a "target name") but no actions list; you either have the named permission or you don't.

The target name is the name of the runtime permission (see below). The naming convention follows the hierarchical property naming convention. Also, an asterisk may appear at the end of the name, following a ".", or by itself, to signify a wildcard match. For example: "loadLibrary.*" or "*" is valid, "*loadLibrary" or "a*b" is not valid.

The following table lists all the possible `RuntimePermission` target names, and for each provides a description of what the permission allows and a discussion of the risks of granting code the permission.

java.lang.RuntimePermission Target Name	What the Permission Allows	Risks of Allowing this Permission
<code>createClassLoader</code>	Creation of a class loader	This is an extremely dangerous permission to grant. Malicious applications that can instantiate their own class loaders could then load their own rogue classes into the system. These newly loaded classes could be placed into any protection domain by the class loader, thereby automatically granting the classes the permissions for that domain.
<code>getClassLoader</code>	Retrieval of a class loader (e.g., the class loader for the calling class)	This would grant an attacker permission to get the class loader for a particular class. This is dangerous because having access to a class's class loader allows the attacker to load other classes available to that class loader. The attacker would typically otherwise not have access to those classes.
<code>setContextClassLoader</code>	Setting of the context class loader used by a thread	The context class loader is used by system code and extensions when they need to lookup resources that might not exist in the system class loader. Granting <code>setContextClassLoader</code> permission would allow code to change which context class loader is used for a particular thread, including system threads.
<code>enableContextClassLoaderOverride</code>	Subclass implementation of the thread context class loader methods	The context class loader is used by system code and extensions when they need to lookup resources that might not exist in the system class loader. Granting <code>enableContextClassLoaderOverride</code> permission would allow a subclass of <code>Thread</code> to override the methods that are used to get or set the context class loader for a particular thread.
<code>closeClassLoader</code>	Closing of a <code>ClassLoader</code>	Granting this permission allows code to close any <code>URLClassLoader</code> that it has a reference to.
<code>setSecurityManager</code>	Setting of the security manager (possibly replacing an existing one)	The security manager is a class that allows applications to implement a security policy. Granting the <code>setSecurityManager</code> permission would allow code to change which security manager is used by installing a different, possibly less restrictive security manager, thereby bypassing checks that would have been enforced by the original security manager.
<code>createSecurityManager</code>	Creation of a new security manager	This gives code access to protected, sensitive methods that may disclose information about other classes or the execution stack.
<code>getenv.{variable name}</code>	Reading of the value of the specified	This would allow code to read the value, or determine the existence, of a particular environment

	environment variable	variable. This is dangerous if the variable contains confidential data.
<code>exitVM.{exit status}</code>	Halting of the Java Virtual Machine with the specified exit status	This allows an attacker to mount a denial-of-service attack by automatically forcing the virtual machine to halt. Note: The "exitVM.*" permission is automatically granted to all code loaded from the application class path, thus enabling applications to terminate themselves. Also, the "exitVM" permission is equivalent to "exitVM.*".
<code>shutdownHooks</code>	Registration and cancellation of virtual-machine shutdown hooks	This allows an attacker to register a malicious shutdown hook that interferes with the clean shutdown of the virtual machine.
<code>setFactory</code>	Setting of the socket factory used by <code>ServerSocket</code> or <code>Socket</code> , or of the stream handler factory used by URL	This allows code to set the actual implementation for the socket, server socket, stream handler, or RMI socket factory. An attacker may set a faulty implementation which mangles the data stream.
<code>setIO</code>	Setting of <code>System.out</code> , <code>System.in</code> , and <code>System.err</code>	This allows changing the value of the standard system streams. An attacker may change <code>System.in</code> to monitor and steal user input, or may set <code>System.err</code> to a "null" <code>OutputStream</code> , which would hide any error messages sent to <code>System.err</code> .
<code>modifyThread</code>	Modification of threads, e.g., via calls to <code>Thread</code> <code>interrupt</code> , <code>stop</code> , <code>suspend</code> , <code>resume</code> , <code>setDaemon</code> , <code>setPriority</code> , <code>setName</code> and <code>setUncaughtExceptionHandler</code> methods	This allows an attacker to modify the behavior of any thread in the system.
<code>stopThread</code>	Stopping of threads via calls to the <code>Thread</code> <code>stop</code> method	This allows code to stop any thread in the system provided that it is already granted permission to access that thread. This poses as a threat, because that code may corrupt the system by killing existing threads.
<code>modifyThreadGroup</code>	Modification of thread groups, e.g., via calls to <code>ThreadGroup</code> <code>destroy</code> , <code>getParent</code> , <code>resume</code> , <code>setDaemon</code> , <code>setMaxPriority</code> , <code>stop</code> , and <code>suspend</code> methods	This allows an attacker to create thread groups and set their run priority.
<code>getProtectionDomain</code>	Retrieval of the <code>ProtectionDomain</code> for a class	This allows code to obtain policy information for a particular code source. While obtaining policy information does not compromise the security of the system, it does give attackers additional information, such as local file names for example, to better aim an attack.
<code>getFileSystemAttributes</code>	Retrieval of file system attributes	This allows code to obtain file system information such as disk usage or disk space available to the caller. This is potentially dangerous because it discloses information about the system hardware configuration and some information about the caller's privilege to write files.
<code>readFileDescriptor</code>	Reading of file descriptors	This would allow code to read the particular file associated with the file descriptor read. This is dangerous if the file contains confidential data.

<code>writeFileDescriptor</code>	Writing to file descriptors	This allows code to write to a particular file associated with the descriptor. This is dangerous because it may allow malicious code to plant viruses or at the very least, fill up your entire disk.
<code>loadLibrary.{library name}</code>	Dynamic linking of the specified library	It is dangerous to allow an applet permission to load native code libraries, because the Java security architecture is not designed to and does not prevent malicious behavior at the level of native code.
<code>accessClassInPackage. {package name}</code>	Access to the specified package via a class loader's <code>loadClass</code> method when that class loader calls the <code>SecurityManager</code> <code>checkPackageAccess</code> method	This gives code access to classes in packages to which it normally does not have access. Malicious code may use these classes to help in its attempt to compromise security in the system.
<code>defineClassInPackage. {package name}</code>	Definition of classes in the specified package, via a class loader's <code>defineClass</code> method when that class loader calls the <code>SecurityManager</code> <code>checkPackageDefinition</code> method.	This grants code permission to define a class in a particular package. This is dangerous because malicious code with this permission may define rogue classes in trusted packages like <code>java.security</code> or <code>java.lang</code> , for example.
<code>accessDeclaredMembers</code>	Warning: <i>Extreme caution should be taken before granting this permission to code</i> , for it provides access to the declared members of a class.	This grants code permission to query a class for its public, protected, default (package) access, and private fields and/or methods. Although the code would have access to the private and protected field and method names, it would not have access to the private/protected field data and would not be able to invoke any private methods. Nevertheless, malicious code may use this information to better aim an attack. Additionally, it may invoke any public methods and/or access public fields in the class. This could be dangerous if the code would normally not be able to invoke those methods and/or access the fields because it can't cast the object to the class/interface with those methods and fields.
<code>queuePrintJob</code>	Initiation of a print job request	This could print sensitive information to a printer, or simply waste paper.
<code>getStackTrace</code>	Retrieval of the stack trace information of another thread.	This allows retrieval of the stack trace information of another thread. This might allow malicious code to monitor the execution of threads and discover vulnerabilities in applications.
<code>setDefaultUncaughtExceptionHandler</code>	Setting the default handler to be used when a thread terminates abruptly due to an uncaught exception.	This allows an attacker to register a malicious uncaught exception handler that could interfere with termination of a thread.
<code>preferences</code>	Represents the permission required to get access to the <code>java.util.prefs.Preferences</code> implementations user or system root which in turn allows retrieval or	This permission allows the user to read from or write to the preferences backing store if the user running the code has sufficient OS privileges to read/write to that backing store. The actual backing store may reside within a traditional filesystem directory or within a registry depending on the platform OS.

17/12/21 8:29Permissions in the JDK

	update operations within the Preferences persistent backing store.	
usePolicy	Granting this permission disables the Java Plug-In's default security prompting behavior.	For more information, refer to Java Plug-In's guides, Applet Security Basics and usePolicy Permission.

NIO-Related Targets

Two NIO-related `RuntimePermission` targets were added in the 1.4 release of the JavaSE JDK:

```
selectorProvider
charsetProvider
```

These `RuntimePermissions` are required to be granted to classes which subclass and implement `java.nio.channel.spi.SelectorProvider` or `java.nio.charset.spi.CharsetProvider`. The permission is checked during invocation of the abstract base class constructor. These permissions ensure trust in classes which implement these security-sensitive provider mechanisms.

See [java.nio.channels.spi.SelectorProvider](#) and [java.nio.channels.spi.CharsetProvider](#) for more information.

NetPermission

A [java.net.NetPermission](#) is for various network permissions. A `NetPermission` contains a name but no actions list; you either have the named permission or you don't.

The following table lists all the possible `NetPermission` target names, and for each provides a description of what the permission allows and a discussion of the risks of granting code the permission.

java.net.NetPermission Target Name	What the Permission Allows	Risks of Allowing this Permission
setDefaultAuthenticator	The ability to set the way authentication information is retrieved when a proxy or HTTP server asks for authentication	Malicious code can set an authenticator that monitors and steals user authentication input as it retrieves the input from the user.
requestPasswordAuthentication	The ability to ask the authenticator registered with the system for a password	Malicious code may steal this password.
specifyStreamHandler	The ability to specify a stream handler when constructing a URL	Malicious code may create a URL with resources that it would normally not have access to (like file:/foo/fum/), specifying a stream handler that gets the actual bytes from someplace it does have access to. Thus it might be able to trick the system into creating a ProtectionDomain/CodeSource for a class even though that class really didn't come from that location.

<code>setProxySelector</code>	The ability to set the proxy selector used to make decisions on which proxies to use when making network connections.	Malicious code can set a ProxySelector that directs network traffic to an arbitrary network host.
<code>getProxySelector</code>	The ability to get the proxy selector used to make decisions on which proxies to use when making network connections.	Malicious code can get a ProxySelector to discover proxy hosts and ports on internal networks, which could then become targets for attack.
<code>setCookieHandler</code>	The ability to set the cookie handler that processes highly security sensitive cookie information for an Http session.	Malicious code can set a cookie handler to obtain access to highly security sensitive cookie information. Some web servers use cookies to save user private information such as access control information, or to track user browsing habit.
<code>getCookieHandler</code>	The ability to get the cookie handler that processes highly security sensitive cookie information for an Http session.	Malicious code can get a cookie handler to obtain access to highly security sensitive cookie information. Some web servers use cookies to save user private information such as access control information, or to track user browsing habit.
<code>setResponseCache</code>	The ability to set the response cache that provides access to a local response cache.	Malicious code getting access to the local response cache could access security sensitive information, or create false entries in the response cache.
<code>getResponseCache</code>	The ability to get the response cache that provides access to a local response cache.	Malicious code getting access to the local response cache could access security sensitive information.

SocketPermission

A [java.net.SocketPermission](https://docs.oracle.com/javase/8/docs/technotes/guides/security/permissions.html) represents access to a network via sockets. A SocketPermission consists of a host specification and a set of "actions" specifying ways to connect to that host. The host is specified as

```
host = (hostname | IPaddress)[:portrange]
portrange = portnumber | -portnumber | portnumber-[portnumber]
```

The host is expressed as a DNS name, as a numerical IP address, or as "localhost" (for the local machine). The wildcard "*" may be included once in a DNS name host specification. If it is included, it must be in the leftmost position, as in "*.sun.com".

The port or portrange is optional. A port specification of the form "N-", where *N* is a port number, signifies all ports numbered *N* and above, while a specification of the form "-N" indicates all ports numbered *N* and below.

The possible ways to connect to the host are

```
accept
connect
listen
resolve
```

The "listen" action is only meaningful when used with "localhost". The "resolve" (resolve host/ip name service lookups) action is implied when any of the other actions are present.

As an example of the creation and meaning of `SocketPermissions`, note that if you have the following entry in your policy file:

```
grant signedBy "mrm" {
    permission java.net.SocketPermission "puffin.example.com:7777", "connect, accept";
};
```

this causes the following permission object to be generated and granted to code signed by "mrm."

```
p1 = new SocketPermission("puffin.example.com:7777", "connect,accept");
```

`p1` represents a permission allowing connections to port 7777 on `puffin.example.com`, and also accepting connections on that port.

Similarly, if you have the following entry in your policy:

```
grant signedBy "paul" {
    permission java.net.SocketPermission "localhost:1024-", "accept, connect, listen";
};
```

this causes the following permission object to be generated and granted to code signed by "paul."

```
p2 = new SocketPermission("localhost:1024-", "accept,connect,listen");
```

`p2` represents a permission allowing accepting connections on, connecting to, or listening on any port between 1024 and 65535 on the local host.

Note: Granting code permission to accept or make connections to remote hosts may be dangerous because malevolent code can then more easily transfer and share confidential data among parties who may not otherwise have access to the data.

URLPermission

Represents permission to access a resource or set of resources defined by a given url, and for a given set of user-settable request methods and request headers. The *name* of the permission is the url string. The *actions* string is a concatenation of the request methods and headers. The range of method and header names is not restricted by this class.

The url

The url string has the following expected structure.

```
scheme : // authority [ / path ]
```

scheme will typically be http or https, but is not restricted by this class. *authority* is specified as:

```
authority = [ userinfo @ ] hostrange [ : portrange ]
portrange = portnumber | -portnumber | portnumber-[portnumber] | *
hostrange = ([*.] dnsname) | IPv4address | IPv6address
```

dnsname is a standard DNS host or domain name, ie. one or more labels separated by ".". *IPv4address* is a standard literal IPv4 address and *IPv6address* is as defined in [RFC 2732](#). Literal IPv6 addresses must however, be enclosed in '[' characters. The *dnsname* specification can be preceded by "*" which means the name will match any hostname whose right-most domain labels are the same as this name. For example, "*.oracle.com" matches "foo.bar.oracle.com"

portrange is used to specify a port number, or a bounded or unbounded range of ports that this permission applies to. If portrange is absent or invalid, then a default port number is assumed if the scheme is `http` (default 80) or `https` (default 443). No default is assumed for other schemes. A wildcard may be specified which means all ports.

userinfo is optional. A userinfo component if present, is ignored when creating a URLPermission, and has no effect on any other methods defined by this class.

The *path* component comprises a sequence of path segments, separated by '/' characters. *path* may also be empty. The path is specified in a similar way to the path in [FilePermission](#). There are three different ways as the following examples show:

URL Examples

Example url	Description
http://www.oracle.com/a/b/c.html	A url which identifies a specific (single) resource
http://www.oracle.com/a/b/*	The '*' character refers to all resources in the same "directory" - in other words all resources with the same number of path components, and which only differ in the final path component, represented by the '*'.
http://www.oracle.com/a/b/-	The '-' character refers to all resources recursively below the preceding path (eg. http://www.oracle.com/a/b/c/d/e.html matches this example).

The '*' and '-' may only be specified in the final segment of a path and must be the only character in that segment. Any query or fragment components of the url are ignored when constructing URLPermissions.

As a special case, urls of the form, "scheme:*" are accepted to mean any url of the given scheme.

The *scheme* and *authority* components of the url string are handled without regard to case. This means `equals(Object)`, `hashCode()` and `implies(Permission)` are case insensitive with respect to these components. If the *authority* contains a literal IP address, then the address is normalized for comparison. The path component is case sensitive.

The actions string

The actions string of a URLPermission is a concatenation of the *method list* and the *request headers list*. These are lists of the permitted request methods and permitted request headers of the permission (respectively). The two lists are separated by a colon ':' character and elements of each list are comma separated. Some examples are:

```
"POST,GET,DELETE"
"GET:X-Foo-Request,X-Bar-Request"
"POST,GET:Header1,Header2"
```

The first example specifies the methods: POST, GET and DELETE, but no request headers. The second example specifies one request method and two headers. The third example specifies two request methods, and two headers.

The colon separator need not be present if the request headers list is empty. No white-space is permitted in the actions string. The action strings supplied to the URLPermission constructors are case-insensitive and are normalized by converting method names to upper-case and header names to the form defines in RFC2616 (lower case with initial letter of each word capitalized). Either list can contain a wild-card "*" character which signifies all request methods or headers respectively.

Note: Depending on the context of use, some request methods and headers may be permitted at all times, and others may not be permitted at any time. For example, the HTTP protocol handler might disallow certain headers such as Content-Length from being set by application code, regardless of whether the security policy in force, permits it.

Note: Granting code permission to access resources on remote hosts may be dangerous because malevolent code can then more easily transfer and share confidential data among parties who may not otherwise have access to the data.

LinkPermission

The `Permission` class for link creation operations.

The following table provides a summary description of what the permission allows, and discusses the risks of granting code the permission.

java.nio.file.LinkPermission Target Name	What the Permission Allows	Risks of Allowing this Permission
hard	Ability to add an existing file to a directory. This is sometimes known as creating a link, or hard link.	Extreme care should be taken when granting this permission. It allows linking to any file or directory in the file system thus allowing the attacker access to all files.
symbolic	Ability to create symbolic links.	Extreme care should be taken when granting this permission. It allows linking to any file or directory in the file system thus allowing the attacker to access to all files.

AllPermission

The `java.security.AllPermission` is a permission that implies all other permissions.

Note: Granting `AllPermission` should be done with extreme care, as it implies all other permissions. Thus, it grants code the ability to run with security disabled. Extreme caution should be taken before granting such a permission to code. This permission should be used only during testing, or in extremely rare cases where an application or applet is completely trusted and adding the necessary permissions to the policy is prohibitively cumbersome.

SecurityPermission

A `java.security.SecurityPermission` is for security permissions. A `SecurityPermission` contains a name (also referred to as a "target name") but no actions list; you either have the named permission or you don't.

The target name is the name of a security configuration parameter (see below). Currently the `SecurityPermission` object is used to guard access to the `Policy`, `Security`, `Provider`, `Signer`, and `Identity` objects.

The following table lists all the possible `SecurityPermission` target names, and for each provides a description of what the permission allows and a discussion of the risks of granting code the permission.

<code>java.security.SecurityPermission</code> Target Name	What the Permission Allows	Risks of Allowing this Permission
<code>createAccessControlContext</code>	Creation of an <code>AccessControlContext</code>	This allows someone to instantiate an <code>AccessControlContext</code> with a <code>DomainCombiner</code> . Extreme care must be taken when granting this permission. Malicious code could create a <code>DomainCombiner</code> that augments the set of permissions granted to code, and even grant the code <code>AllPermission</code> .
<code>getDomainCombiner</code>	Retrieval of an <code>AccessControlContext</code> 's <code>DomainCombiner</code>	This allows someone to query the policy via the <code>getPermissions</code> call, which discloses which permissions would be granted to a given <code>CodeSource</code> . While revealing the policy does not compromise the security of the system, it does provide malicious code with additional information which it may use to better aim an attack. It is wise not to divulge more information than necessary.
<code>getPolicy</code>	Retrieval of the system-wide security policy (specifically, of the currently-installed Policy object)	This allows someone to query the policy via the <code>getPermissions</code> call, which discloses which permissions would be granted to a given <code>CodeSource</code> . While revealing the policy does not compromise the security of the system, it does provide malicious code with additional information which it may use to better aim an attack. It is wise not to divulge more information than necessary.
<code>setPolicy</code>	Setting of the system-wide security policy (specifically, the Policy object)	Granting this permission is extremely dangerous, as malicious code may grant itself all the necessary permissions it needs to successfully mount an attack on the system.
<code>createPolicy.{policy type}</code>	Getting an instance of a Policy via <code>Policy.getInstance</code>	Granting this permission enables code to obtain a Policy object. Malicious code may query the Policy object to determine what permissions have been granted to code other than itself.
<code>getProperty.{key}</code>	Retrieval of the security property with the specified key	Depending on the particular key for which access has been granted, the code may have access to the list of security providers, as well as the location of the system-wide and user security policies. While revealing this information does not compromise the security of the system, it does provide malicious code with additional information which it may use to better aim an attack.
<code>setProperty.{key}</code>	Setting of the security property with the specified key	This could include setting a security provider or defining the location of the system-wide security policy. Malicious code that has permission to set a new security provider may set a rogue provider that steals confidential information such as cryptographic private keys. In addition, malicious code with permission to set the location of the system-wide security policy may point it to a security policy that grants the attacker all the necessary permissions it requires to successfully mount an attack on the system.
<code>insertProvider</code>	Addition of a new provider	This would allow somebody to introduce a possibly malicious provider (e.g., one that discloses the private keys passed to it) as the highest-priority provider. This would be possible because the Security object (which manages the installed providers) currently does not check the integrity or authenticity of a provider before attaching it. The "insertProvider" permission subsumes the "insertProvider.{provider name}" permission (see the section below for more information).

<code>removeProvider.{provider name}</code>	Removal of the specified provider	This may change the behavior or disable execution of other parts of the program. If a provider subsequently requested by the program has been removed, execution may fail. Also, if the removed provider is not explicitly requested by the rest of the program, but it would normally be the provider chosen when a cryptography service is requested (due to its previous order in the list of providers), a different provider will be chosen instead, or no suitable provider will be found, thereby resulting in program failure.
<code>clearProviderProperties. {provider name}</code>	"Clearing" of a Provider so that it no longer contains the properties used to look up services implemented by the provider	This disables the lookup of services implemented by the provider. This may thus change the behavior or disable execution of other parts of the program that would normally utilize the Provider, as described under the "removeProvider.{provider name}" permission.
<code>putProviderProperty.{provider name}</code>	Setting of properties for the specified Provider	The provider properties each specify the name and location of a particular service implemented by the provider. By granting this permission, you let code replace the service specification with another one, thereby specifying a different implementation.
<code>removeProviderProperty.{provider name}</code>	Removal of properties from the specified Provider	This disables the lookup of services implemented by the provider. They are no longer accessible due to removal of the properties specifying their names and locations. This may change the behavior or disable execution of other parts of the program that would normally utilize the Provider, as described under the "removeProvider.{provider name}" permission.

The following permissions are associated with classes that have been deprecated: `Identity`, `IdentityScope`, `Signer`. Use of them is discouraged. See the applicable classes for more information.

<code>java.security.SecurityPermission</code> Target Name	What the Permission Allows	Risks of Allowing this Permission
<code>insertProvider.{provider name}</code>	Addition of a new provider, with the specified name	This would allow somebody to introduce a possibly malicious provider (e.g., one that discloses the private keys passed to it) as the highest-priority provider. This would be possible because the Security object (which manages the installed providers) currently does not check the integrity or authenticity of a provider before attaching it.
<code>setSystemScope</code>	Setting of the system identity scope	This would allow an attacker to configure the system identity scope with certificates that should not be trusted, thereby granting applet or application code signed with those certificates privileges that would have been denied by the system's original identity scope.
<code>setIdentityPublicKey</code>	Setting of the public key for an Identity	If the identity is marked as "trusted", this allows an attacker to introduce a different public key (e.g., its own) that is not trusted by the system's identity scope, thereby granting applet or application code signed with that public key privileges that would have been denied otherwise.
<code>setIdentityInfo</code>	Setting of a general information string for an Identity	This allows attackers to set the general description for an identity. This may trick applications into using a different identity than intended or may prevent applications from finding a particular identity.
<code>addIdentityCertificate</code>	Addition of a certificate for	This allows attackers to set a certificate for an identity's public key. This is dangerous because it affects the trust

	an Identity	relationship across the system. This public key suddenly becomes trusted to a wider audience than it otherwise would be.
<code>removeIdentityCertificate</code>	Removal of a certificate for an Identity	This allows attackers to remove a certificate for an identity's public key. This is dangerous because it affects the trust relationship across the system. This public key suddenly becomes considered less trustworthy than it otherwise would be.
<code>printIdentity</code>	Viewing the name of a principal and optionally the scope in which it is used, and whether or not it is considered "trusted" in that scope.	The scope that is printed out may be a filename, in which case it may convey local system information. For example, here's a sample printout of an identity named "carol", who is marked not trusted in the user's identity database: carol[/home/luehe/identitydb.obj][not trusted]
<code>getSignerPrivateKey</code>	Retrieval of a Signer's private key	It is very dangerous to allow access to a private key; private keys are supposed to be kept secret. Otherwise, code can use the private key to sign various files and claim the signature came from the Signer.
<code>setSignerKeyPair</code>	Setting of the key pair (public key and private key) for a Signer	This would allow an attacker to replace somebody else's (the "target's") keypair with a possibly weaker keypair (e.g., a keypair of a smaller keysize). This also would allow the attacker to listen in on encrypted communication between the target and its peers. The target's peers might wrap an encryption session key under the target's "new" public key, which would allow the attacker (who possesses the corresponding private key) to unwrap the session key and decipher the communication data encrypted under that session key.

UnresolvedPermission

The `java.security.UnresolvedPermission` class is used to hold `Permissions` that were "unresolved" when the `Policy` was initialized. An unresolved permission is one whose actual `Permission` class does not yet exist at the time the `Policy` is initialized (see below).

The policy for a Java runtime (specifying which permissions are available for code from various principals) is represented by a `Policy` object. Whenever a `Policy` is initialized or refreshed, `Permission` objects of appropriate classes are created for all permissions allowed by the `Policy`.

Many permission class types referenced by the policy configuration are ones that exist locally (i.e., ones that can be found on CLASSPATH). Objects for such permissions can be instantiated during `Policy` initialization. For example, it is always possible to instantiate a `java.io.FilePermission`, since the `FilePermission` class is found on the CLASSPATH.

Other permission classes may not yet exist during `Policy` initialization. For example, a referenced permission class may be in a JAR file that will later be loaded. For each such class, an `UnresolvedPermission` is instantiated. Thus, an `UnresolvedPermission` is essentially a "placeholder" containing information about the permission.

Later, when code calls `AccessController.checkPermission` on a permission of a type that was previously unresolved, but whose class has since been loaded, previously-unresolved permissions of that type are "resolved". That is, for each such `UnresolvedPermission`, a new object of the appropriate class type is instantiated, based on the information in the `UnresolvedPermission`. This new object replaces the `UnresolvedPermission`, which is removed.

SQLPermission

The permission for which the `SecurityManager` will check when code that is running in an applet, or an application with an instance of `SecurityManager` enabled, calls one of the following methods:

- `java.sql.DriverManager.setLogWriter`
- `java.sql.DriverManager.setLogStream` (deprecated)
- `javax.sql.rowset.spi.SyncFactory.setJNDIContext`
- `javax.sql.rowset.spi.SyncFactory.setLogger`
- `java.sql.Connection.setNetworktimeout`
- `java.sql.Connection.abort`

If there is no `SQLPermission` object, these methods throw a `java.lang.SecurityException` as a runtime exception.

A `SQLPermission` object contains a name (also referred to as a "target name") but no actions list; there is either a named permission or there is not. The target name is the name of the permission (see the following table that lists all the possible `SQLPermission` names). The naming convention follows the hierarchical property naming convention. In addition, an asterisk (*) may appear at the end of the name, following a dot (.), or by itself, to signify a wildcard match. For example: `loadLibrary.*` or `*` is valid, but `*loadLibrary` or `a*b` is not valid.

The following table lists all the possible `SQLPermission` target names. The table gives a description of what the permission allows and a discussion of the risks of granting code the permission.

<code>java.sql.SQLPermission</code> Target Name	What Permission Allows	Risks of Allowing This Permission
<code>setLog</code>	Setting of the logging stream	This is a dangerous permission to grant. The contents of the log can contain usernames and passwords, SQL statements, and SQL data.
<code>callAbort</code>	Invocation of the <code>Connection</code> method <code>abort</code>	Permits an application to terminate a physical connection to a database.
<code>setSyncFactory</code>	Invocation of the <code>SyncFactory</code> methods <code>setJNDIContext</code> and <code>setLogger</code>	Permits an application to specify the JNDI context from which the <code>SyncProvider</code> implementations can be retrieved from and the logging object to be used by the <code>SyncProvider</code> implementation.
<code>setNetworkTimeout</code>	Invocation of the <code>Connection</code> method <code>setNetworkTimeout</code>	Permits an application to specify the maximum period a <code>Connection</code> or objects created from the <code>Connection</code> object will wait for the database to reply to any one request.
<code>deregisterDriver</code>	Allows the invocation of the <code>DriverManager</code> method <code>deregisterDriver</code> .	Permits an application to remove a JDBC driver from the list of registered Drivers and release its resources.

LoggingPermission

A `SecurityManager` will check the `java.util.logging.LoggingPermission` object when code running with a `SecurityManager` calls one of the logging control methods (such as `Logger.setLevel`).

Currently there is only one named `LoggingPermission`, "control". `control` grants the ability to control the logging configuration; for example by adding or removing Handlers, by adding or removing Filters, or by changing logging levels.

Normally you do not create `LoggingPermission` objects directly; instead they are created by the security policy code based on reading the security policy file.

PropertyPermission

A `java.util.PropertyPermission` is for property permissions.

The name is the name of the property ("java.home", "os.name", etc). The naming convention follows the hierarchical property naming convention. Also, an asterisk may appear at the end of the name, following a ".", or by itself, to signify a wildcard match. For example: "java.*" or "*" is valid, "java" or "a*b" is not valid.

The actions to be granted are passed to the constructor in a string containing a list of zero or more comma-separated keywords. The possible keywords are "read" and "write". Their meaning is defined as follows:

Keyword	Meaning
read	Permission to read. Allows <code>System.getProperty</code> to be called.
write	Permission to write. Allows <code>System.setProperty</code> to be called.

The actions string is converted to lowercase before processing.

Care should be taken before granting code permission to access certain system properties. For example, granting permission to access the "java.home" system property gives potentially malevolent code sensitive information about the system environment (the location of the runtime environment's directory). Also, granting permission to access the "user.name" and "user.home" system properties gives potentially malevolent code sensitive information about the user environment (the user's account name and home directory).

MBeanPermission

Permission controlling access to `MBeanServer` operations. If a security manager has been set using `System.setSecurityManager(java.lang.SecurityManager)`, most operations on the `MBeanServer` require that the caller's permissions imply an `MBeanPermission` appropriate for the operation. This is described in detail in the documentation for the `MBeanServer` interface.

As with other `Permission` objects, an `MBeanPermission` can represent either a permission that you *have* or a permission that you *need*. When a sensitive operation is being checked for permission, an `MBeanPermission` is constructed representing the permission you need. The operation is only allowed if the permissions you have *imply* the permission you need.

An `MBeanPermission` contains four items of information:

- The *action*. For a permission you need, this is one of the actions in the list [below](#). For a permission you have, this is a comma-separated list of those actions, or *, representing all actions.

The action is returned by `getActions()`.

- The *class name*.

For a permission you need, this is the class name of an `MBean` you are accessing, as returned by `MBeanServer.getMBeanInfo(name).getClassName()`. Certain operations do not reference a class name, in which case the class name is null.

For a permission you have, this is either empty or a *class name pattern*. A class name pattern is a string following the Java conventions for dot-separated class names. It may end with `".*"` meaning that the permission grants access to any class that begins with the string preceding `".*"`. For instance, `"javax.management.*"` grants access to `javax.management.MBeanServerDelegate` and `javax.management.timer.Timer`, among other classes.

A class name pattern can also be empty or the single character `"*"`, both of which grant access to any class.

- The *member*.

For a permission you need, this is the name of the attribute or operation you are accessing. For operations that do not reference an attribute or operation, the member is null.

For a permission you have, this is either the name of an attribute or operation you can access, or it is empty or the single character `"*"`, both of which grant access to any member.

- The *object name*.

For a permission you need, this is the `ObjectName` of the `MBean` you are accessing. For operations that do not reference a single `MBean`, it is null. It is never an object name pattern.

For a permission you have, this is the `ObjectName` of the `MBean` or `MBeans` you can access. It may be an object name pattern to grant access to all `MBeans` whose names match the pattern. It may also be empty, which grants access to all `MBeans` whatever their name.

If you have an `MBeanPermission`, it allows operations only if all four of the items match.

The class name, member, and object name can be written together as a single string, which is the *name* of this permission. The name of the permission is the string returned by `getName()`. The format of the string is:

```
className#member[objectName]
```

The object name is written using the usual syntax for `ObjectName`. It may contain any legal characters, including `]`. It is terminated by a `]` character that is the last character in the string.

One or more of the `className`, `member`, or `objectName` may be omitted. If the `member` is omitted, the `#` may be too (but does not have to be). If the `objectName` is omitted, the `[]` may be too (but does not have to be). It is not legal to omit all three items, that is to have a *name* that is the empty string.

One or more of the `className`, `member`, or `objectName` may be the character `"-"`, which is equivalent to a null value. A null value is implied by any value (including another null value) but does not imply any other value.

The possible actions are these:

- `addNotificationListener`

- `getAttribute`
- `getClassLoader`
- `getClassLoaderFor`
- `getClassLoaderRepository`
- `getDomains`
- `getMBeanInfo`
- `getObjectInstance`
- `instantiate`
- `invoke`
- `isInstanceOf`
- `queryMBeans`
- `queryNames`
- `registerMBean`
- `removeNotificationListener`
- `setAttribute`
- `unregisterMBean`

In a comma-separated list of actions, spaces are allowed before and after each action.

`MBeanServerPermission`

A Permission to perform actions related to `MBeanServers`. The *name* of the permission specifies the operation requested or granted by the permission. For a granted permission, it can be `*` to allow all of the `MBeanServer` operations specified below. Otherwise, for a granted or requested permission, it must be one of the following:

`createMBeanServer`

Create a new `MBeanServer` object using the method `MBeanServerFactory.createMBeanServer()` or `MBeanServerFactory.createMBeanServer(java.lang.String)`.

`findMBeanServer`

Find an `MBeanServer` with a given name, or all `MBeanServers` in this JVM, using the method `MBeanServerFactory.findMBeanServer(java.lang.String)`.

`newMBeanServer`

Create a new `MBeanServer` object without keeping a reference to it, using the method `MBeanServerFactory.newMBeanServer()` or `MBeanServerFactory.newMBeanServer(java.lang.String)`.

`releaseMBeanServer`

Remove the `MBeanServerFactory`'s reference to an `MBeanServer`, using the method `MBeanServerFactory.releaseMBeanServer(javax.management.MBeanServer)`.

The *name* of the permission can also denote a list of one or more comma-separated operations. Spaces are allowed at the beginning and end of the *name* and before and after commas.

`MBeanServerPermission("createMBeanServer")` implies `MBeanServerPermission("newMBeanServer")`.

MBeanTrustPermission

This permission represents "trust" in a signer or codebase.

`MBeanTrustPermission` contains a target name but no actions list. A single target name, "register", is defined for this permission. The target "*" is also allowed, permitting "register" and any future targets that may be defined. Only the null value or the empty string are allowed for the action to allow the policy object to create the permissions specified in the policy file.

If a signer, or codesource is granted this permission, then it is considered a trusted source for `MBeans`. Only `MBeans` from trusted sources may be registered in the `MBeanServer`.

SubjectDelegationPermission

Permission required by an authentication identity to perform operations on behalf of an authorization identity.

A `SubjectDelegationPermission` contains a name (also referred to as a "target name") but no actions list; you either have the named permission or you don't.

The target name is the name of the authorization principal classname followed by a period and the authorization principal name, that is "*PrincipalClassName.PrincipalName*".

An asterisk may appear by itself, or if immediately preceded by a "." may appear at the end of the target name, to signify a wildcard match.

For example, "*", "javax.management.remote.JMXPrincipal.*" and "javax.management.remote.JMXPrincipal.delegate" are valid target names. The first one denotes any principal name from any principal class, the second one denotes any principal name of the concrete principal class `javax.management.remote.JMXPrincipal` and the third one denotes a concrete principal name `delegate` of the concrete principal class `javax.management.remote.JMXPrincipal`.

SSLPermission

The `javax.net.ssl.SSLPermission` class is for various network permissions. An `SSLPermission` contains a name (also referred to as a "target name") but no actions list; you either have the named permission or you don't.

The target name is the name of the network permission (see below). The naming convention follows the hierarchical property naming convention. Also, an asterisk may appear at the end of the name, following a ".", or by itself, to signify a wildcard match. For example: "foo.*" or "*" is valid, "*foo" or "a*b" is not valid.

The following table lists all the possible `SSLPermission` target names, and for each provides a description of what the permission allows and a discussion of the risks of granting code the permission.

Permission Target Name	What the Permission Allows	Risks of Allowing this Permission
setHostnameVerifier	The ability to set a callback which can decide whether to allow a mismatch between the host being connected to by an <code>HttpsURLConnection</code> and the common name field in server certificate.	Malicious code can set a verifier that monitors host names visited by <code>HttpsURLConnection</code> requests or that allows server certificates with invalid common names.
getSSLSessionContext	The ability to get the <code>SSLSessionContext</code> of an <code>SSLSession</code> .	Malicious code may monitor sessions which have been established with SSL peers or

		might invalidate sessions to slow down performance.
<code>setDefaultSSLContext</code>	The ability to set the default SSL context.	When applications use default <code>SSLContext</code> , by setting the default SSL context, malicious code may use unproved trust material, key material and random generator, or use dangerous SSL socket factory and SSL server socket factory.

AuthPermission

The `javax.security.auth.AuthPermission` class is for authentication permissions. An `AuthPermission` contains a name (also referred to as a "target name") but no actions list; you either have the named permission or you don't.

Currently the `AuthPermission` object is used to guard access to the `Subject`, `SubjectDomainCombiner`, `LoginContext` and `Configuration` objects.

The following table lists all the possible `AuthPermission` target names, and for each provides a description of what the permission allows and a discussion of the risks of granting code the permission.

Permission Target Name	What the Permission Allows	Risks of Allowing this Permission
<code>doAs</code>	Invocation of the <code>Subject.doAs</code> methods	This enables an application to invoke code (Actions) under the identity of any <code>Subject</code> specified to the <code>doAs</code> method.
<code>doAsPrivileged</code>	Invocation of the <code>Subject.doAsPrivileged</code> methods	This enables an application to invoke code (Actions) under the identity of any <code>Subject</code> specified to the <code>doAsPrivileged</code> method. Additionally, the caller may remove itself from the call stack (and hence from subsequent security decisions) if it passes <code>null</code> as the <code>AccessControlContext</code> .
<code>getSubject</code>	Retrieving the <code>Subject</code> from the provided <code>AccessControlContext</code>	This permits an application to gain access to an authenticated <code>Subject</code> . The application can then access the <code>Subject</code> 's authenticated Principals and public credentials.
<code>getSubjectFromDomainCombiner</code>	Retrieving the <code>Subject</code> from a <code>SubjectDomainCombiner</code>	This permits an application to gain access to the authenticated <code>Subject</code> associated with a <code>SubjectDomainCombiner</code> . The application can then access the <code>Subject</code> 's authenticated Principals and public credentials.
<code>setReadOnly</code>	Setting a <code>Subject</code> read-only	This permits an application to set a <code>Subject</code> 's <code>Principal</code> , public credential and private credential sets to be read-only. This can be potentially used as a type of denial of service attack.
<code>modifyPrincipals</code>	Make modifications to a <code>Subject</code> 's <code>Principal</code> set	Access control decisions are based on the Principals associated with a <code>Subject</code> . This permission permits an application to make any modifications to a <code>Subject</code> 's <code>Principal</code> set, thereby affecting subsequent security decisions.
<code>modifyPublicCredentials</code>	Make modifications to a <code>Subject</code> 's public credential set	This permission permits an application to add or remove public credentials from a <code>Subject</code> . This may affect code that relies on the proper set of private credentials to exist in that <code>Subject</code> .

<code>modifyPrivateCredentials</code>	Make modifications to a Subject's private credential set	This permission permits an application to add or remove private credentials from a <code>Subject</code> . This may affect code that relies on the proper set of private credentials to exist in that <code>Subject</code> .
<code>refreshCredential</code>	Refresh a credential <code>Object</code> that implements the <code>Refreshable</code> interface	This permission permits an application to refresh a credential that is intended to expire.
<code>destroyCredential</code>	Destroy a credential <code>Object</code> that implements the <code>Destroyable</code> interface	This permission permits an application to potentially destroy a credential as a denial of service attack.
<code>createLoginContext.{name}</code>	Instantiate a <code>LoginContext</code> with the specified <i>name</i>	For security purposes, an administrator might not want an application to be able to authenticate to any <code>LoginModule</code> . This permission permits an application to authenticate to the <code>LoginModules</code> configured for the specified <i>name</i> .
<code>getLoginConfiguration</code>	Retrieve the system-wide login <code>Configuration</code>	Allows an application to determine all the <code>LoginModules</code> that are configured for every application in the system.
<code>setLoginConfiguration</code>	Set the system-wide login <code>Configuration</code>	Allows an application to configure the <code>LoginModules</code> for every application in the system.
<code>createLoginConfiguration.{configuration type}</code>	Obtain a <code>Configuration</code> object via <code>Configuration.getInstance</code>	Allows an application to see all the <code>LoginModules</code> that are specified in the configuration.
<code>refreshLoginConfiguration</code>	Refresh the system-wide login <code>Configuration</code>	Allows an application to refresh the login <code>Configuration</code> .

DelegationPermission

The `javax.security.auth.kerberos.DelegationPermission` class is used to restrict the usage of the Kerberos delegation model; ie, forwardable and proxiable tickets.

The target name of this Permission specifies a pair of kerberos service principals. The first is the subordinate service principal being entrusted to use the Ticket Granting Ticket (TGT). The second service principal designates the target service the subordinate service principal is to interact with on behalf of the initiating `KerberosPrincipal`. This latter service principal is specified to restrict the use of a proxiable ticket.

For example, to specify the "host" service use of a forwardable TGT, the target permission is specified as follows:

```
DelegationPermission("\"host/foo.example.com@EXAMPLE.COM\" \"krbtgt/EXAMPLE.COM@EXAMPLE.COM\"");
```

To give the "backup" service a proxiable NFS service ticket, the target permission might be specified:

```
DelegationPermission("\"backup/bar.example.com@EXAMPLE.COM\" \"nfs/home.EXAMPLE.COM@EXAMPLE.COM\"");
```

ServicePermission

The `javax.security.auth.kerberos.ServicePermission` class is used to protect Kerberos services and the credentials necessary to access those services. There is a one to one mapping of a service principal and the credentials necessary to access the service. Therefore granting access to a service principal implicitly grants access to the credential necessary to establish a security context with the service principal. This applies regardless of whether the credentials are in a cache or acquired via an exchange with the KDC. The credential can be either a ticket granting ticket, a service ticket or a secret key from a key table.

A `ServicePermission` contains a service principal name and a list of actions which specify the context the credential can be used within.

The service principal name is the canonical name of the `KereberosPrincipal` supplying the service, that is the `KerberosPrincipal` represents a Kerberos service principal. This name is treated in a case sensitive manner.

Granting this permission implies that the caller can use a cached credential (Ticket Granting Ticket, service ticket or secret key) within the context designated by the action. In the case of the TGT, granting this permission also implies that the TGT can be obtained by an `Authentication Service` exchange.

The possible actions are:

Action	Meaning
<code>initiate</code>	Allows the caller to use the credential to initiate a security context with a service principal.
<code>accept</code>	Allows the caller to use the credential to accept security context as a particular principal.

For example, to specify the permission to access to the TGT to initiate a security context the permission is constructed as follows:

```
ServicePermission("krbtgt/EXAMPLE.COM@EXAMPLE.COM", "initiate");
```

To obtain a service ticket to initiate a context with the "host" service the permission is constructed as follows:

```
ServicePermission("host/foo.example.com@EXAMPLE.COM", "initiate");
```

For a Kerberized server the action is "accept". For example, the permission necessary to access and use the secret key of the Kerberized "host" service (telnet and the likes) would be constructed as follows:

```
ServicePermission("host/foo.example.com@EXAMPLE.COM", "accept");
```

PrivateCredentialPermission

The `javax.security.auth.PrivateCredentialPermission` class is used to protect access to private Credentials belonging to a particular `Subject`. The `Subject` is represented by a Set of Principals.

The target name of this `Permission` specifies a `Credential` class name, and a Set of Principals. The only valid value for this `Permission`'s actions is, "read". The target name must abide by the following syntax:

```
CredentialClass {PrincipalClass "PrincipalName"}*
```

For example, the following permission grants access to the `com.sun.PrivateCredential` owned by `Subjects` which have a `com.sun.Principal` with the name, "duke".

Note: Although this example, as well as all the examples below, do not contain `Codebase`, `SignedBy`, or `Principal` information in the grant statement (for simplicity reasons), actual policy configurations should specify that information when appropriate.

```
grant {
    permission javax.security.auth.PrivateCredentialPermission
        "com.sun.PrivateCredential com.sun.Principal \"duke\"",
        "read";
};
```

If `CredentialClass` is `"*"`, then access is granted to all private `Credentials` belonging to the specified `Subject`. If `PrincipalName` is `"*"`, then access is granted to the specified `Credential` owned by any `Subject` that has the specified `Principal` (the actual `PrincipalName` doesn't matter). For example, the following grants access to the `a.b.Credential` owned by any `Subject` that has an `a.b.Principal`.

```
grant {
    permission javax.security.auth.PrivateCredentialPermission
        "a.b.Credential a.b.Principal \"*\"",
        "read";
};
```

If both the `PrincipalClass` and `"PrincipalName"` are `"*"`, then access is granted to the specified `Credential` owned by any `Subject`. In addition, the `PrincipalClass/PrincipalName` pairing may be repeated:

```
grant {
    permission javax.security.auth.PrivateCredentialPermission
        "a.b.Credential a.b.Principal \"duke\" c.d.Principal \"dukette\"",
        "read";
};
```

The above code grants access to the private `Credential`, `"a.b.Credential"`, belonging to a `Subject` with at least two associated `Principals`: `"a.b.Principal"` with the name, `"duke"`, and `"c.d.Principal"`, with the name, `"dukette"`.

AudioPermission

The `javax.sound.sampled.AudioPermission` class represents access rights to the audio system resources. An `AudioPermission` contains a target name but no actions list; you either have the named permission or you don't.

The target name is the name of the audio permission (see the table below). The names follow the hierarchical property-naming convention. Also, an asterisk can be used to represent all the audio permissions.

The following table lists the possible `AudioPermission` target names. For each name, the table provides a description of exactly what that permission allows, as well as a discussion of the risks of granting code the permission.

Permission Target	What the Permission Allows	Risks of Allowing this Permission
-------------------	----------------------------	-----------------------------------

17/12/21 8:29Permissions in the JDK

Name		
play	Audio playback through the audio device or devices on the system. Allows the application to obtain and manipulate lines and mixers for audio playback (rendering).	In some cases use of this permission may affect other applications because the audio from one line may be mixed with other audio being played on the system, or because manipulation of a mixer affects the audio for all lines using that mixer.
record	Audio recording through the audio device or devices on the system. Allows the application to obtain and manipulate lines and mixers for audio recording (capture).	In some cases use of this permission may affect other applications because manipulation of a mixer affects the audio for all lines using that mixer. This permission can enable an applet or application to eavesdrop on a user.

JAXBPermission

This class is for JAXB permissions. A `JAXBPermission` contains a name (also referred to as a "target name") but no actions list; you either have the named permission or you don't. The target name is the name of the JAXB permission (see below).

The following table lists all the possible `JAXBPermission` target names, and for each provides a description of what the permission allows and a discussion of the risks of granting code the permission.

javax.xml.bind.JAXBPermission Target Name	What the Permission Allows	Risks of Allowing this Permission
setDatatypeConverter	Allows the code to set VM-wide <code>DatatypeConverterInterface</code> via the <code>setDatatypeConverter</code> method that all the methods on <code>DatatypeConverter</code> uses.	Malicious code can set <code>DatatypeConverterInterface</code> , which has VM-wide singleton semantics, before a genuine JAXB implementation sets one. This allows malicious code to gain access to objects that it may otherwise not have access to, such as <code>Frame.getFrames()</code> that belongs to another application running in the same JVM.

WebServicePermission

This class defines web service permissions.

Web service Permissions are identified by name (also referred to as a "target name") alone. There are no actions associated with them.

The following permission target name is defined:

publishEndpoint

The `publishEndpoint` permission allows publishing a web service endpoint using the `publish` methods defined by the `javax.xml.ws.Endpoint` class.

Granting `publishEndpoint` allows the application to be exposed as a network service. Depending on the security of the runtime and the security of the application, this may introduce a security hole that is remotely exploitable.

Methods and the Permissions They Require

The following table contains a list of all the methods that require permissions, and for each tells which `SecurityManager` method it calls and which permission is checked for by the default implementation of that `SecurityManager` method.

Note: This list is not complete and does not include several new methods that require permissions that have been introduced in recent versions of the JDK. See the [API Specification](#) for additional information on methods that throw `SecurityException` and the permissions that are required.

Thus, with the default `SecurityManager` method implementations, a call to a method in the left-hand column can only be successful if the permission specified in the corresponding entry in the right-hand column is allowed by the policy currently in effect. For example, the following row:

Method	SecurityManager Method Called	Permission
<code>java.awt.Toolkit getSystemEventQueue();</code>	<code>checkPermission</code>	<code>java.awt.AWTPermission "accessEventQueue";</code>

specifies that a call to the `getSystemEventQueue` method in the `java.awt.Toolkit` class results in a call to the `checkPermission` `SecurityManager` method, which can only be successful if the following permission is granted to code on the call stack:

```
java.awt.AWTPermission "accessEventQueue";
```

The convention of:

Method	SecurityManager Method Called	Permission
<code>some.package.class public static void someMethod(String foo);</code>	<code>checkXXX</code>	<code>SomePermission "{foo}";</code>

means the runtime value of `foo` replaces the string `{foo}` in the permission name.

As an example, here is one table entry:

Method	SecurityManager Method Called	Permission
<code>java.io.FileInputStream FileInputStream(String name)</code>	<code>checkRead(String)</code>	<code>java.io.FilePermission "{name}", "read";</code>

If the `FileInputStream` method (in this case, a constructor) is called with `"/test/MyTestFile"` as the `name` argument, as in

```
FileInputStream("/test/MyTestFile");
```

then in order for the call to succeed, the following permission must be set in the current policy, allowing read access to the file `"/test/MyTestFile"`:

```
java.io.FilePermission "/test/MyTestFile", "read";
```

More specifically, the permission must either be explicitly set, as above, or implied by another permission, such as the following:

```
java.io.FilePermission "/test/*", "read";
```

which allows read access to any files in the `"/test"` directory.

In some cases, a term in braces is not exactly the same as the name of a specific method argument but is meant to represent the relevant value. Here is an example:

Method	SecurityManager Method Called	Permission
<pre>java.net.DatagramSocket public synchronized void receive(DatagramPacket p);</pre>	<pre>checkAccept({host}, {port})</pre>	<pre>java.net.SocketPermission "{host}:{port}", "accept";</pre>

Here, the appropriate host and port values are calculated by the `receive` method and passed to `checkAccept`.

In most cases, just the name of the SecurityManager method called is listed. Where the method is one of multiple methods of the same name, the argument types are also listed, for example for `checkRead(String)` and `checkRead(FileDescriptor)`. In other cases where arguments may be relevant, they are also listed.

The following table is ordered by package name. That is, the methods in classes in the `java.awt` package are listed first, followed by methods in classes in the `java.io` package, and so on.

Methods and the Permissions They Require

Method	SecurityManager Method Called	Permission
<pre>java.awt.Graphics2d public abstract void setComposite(Composite comp)</pre>	<pre>checkPermission</pre>	<pre>java.awt.AWTPermission "readDisplayPixels" if this Graphics2D context is drawing to a Component on the display screen and the Composite is a custom object rather than an instance of the AlphaComposite class. Note: The setComposite method is actually abstract and thus can't invoke security checks. Each actual implementation of the method should call the java.lang.SecurityManager checkPermission method with a java.awt.AWTPermission("readDisplayPixels") permission under the conditions noted.</pre>
<pre>java.awt.Robot public Robot() public Robot(GraphicsDevice screen)</pre>	<pre>checkPermission</pre>	<pre>java.awt.AWTPermission "createRobot"</pre>
<pre>java.awt.Toolkit public void addAWTEventListener(</pre>	<pre>checkPermission</pre>	<pre>java.awt.AWTPermission "listenToAllAWTEvents"</pre>

<pre> AWTEventListener listener, long eventMask) public void removeAWTEventListener(AWTEventListener listener) </pre>		
<pre> java.awt.Toolkit public abstract PrintJob getPrintJob(Frame frame, String jobtitle, Properties props) </pre>	checkPrintJobAccess	java.lang.RuntimePermission "queuePrintJob" Note: The getPrintJob method is actually abstract and thus can't invoke security checks. Each actual implementation of the method should call the java.lang.SecurityManager checkPrintJobAccess method, which is successful only if the java.lang.RuntimePermission "queuePrintJob" permission is currently allowed.
<pre> java.awt.Toolkit public abstract Clipboard getSystemClipboard() </pre>	checkPermission	java.awt.AWTPermission "accessClipboard" Note: The getSystemClipboard method is actually abstract and thus can't invoke security checks. Each actual implementation of the method should call the checkPermission method, which is successful only if the java.awt.AWTPermission "accessClipboard" permission is currently allowed.
<pre> java.awt.Toolkit public final EventQueue getSystemEventQueue() </pre>	checkPermission	java.awt.AWTPermission "accessEventQueue"
<pre> java.awt.Window Window() </pre>	checkPermission	If java.awt.AWTPermission "showWindowWithoutWarningBanner" is set, the window will be displayed without a banner warning that the window was created by an applet. If it's not set, such a banner will be displayed.
<pre> java.beans.Beans public static void setDesignTime(boolean isDesignTime) public static void setGuiAvailable(boolean isGuiAvailable) </pre>	checkPropertiesAccess	java.util.PropertyPermission "*", "read,write"

<pre> java.beans.Introspector public static synchronized void setBeanInfoSearchPath(String path[]) java.beans.PropertyEditorManager public static void registerEditor(Class targetType, Class editorClass) public static synchronized void setEditorSearchPath(String path[]) </pre>		
<pre> java.io.File public boolean delete() public void deleteOnExit() </pre>	checkDelete(String)	java.io.FilePermission "{name}", "delete"
<pre> java.io.FileInputStream FileInputStream(FileDescriptor fdObj) </pre>	checkRead(FileDescriptor)	java.lang.RuntimePermission "readFileDescriptor"
<pre> java.io.FileInputStream FileInputStream(String name) FileInputStream(File file) java.io.File public boolean exists() public boolean canRead() public boolean isFile() public boolean isDirectory() public boolean isHidden() public long lastModified() public long length() public String[] list() public String[] list(FilenameFilter filter) public File[] listFiles() public File[] listFiles(FilenameFilter filter) public File[] listFiles(FileFilter filter) java.io.RandomAccessFile </pre>	checkRead(String)	java.io.FilePermission "{name}", "read"

<pre> RandomAccessFile(String name, String mode) RandomAccessFile(File file, String mode) (where mode is "r" in both of these) </pre>		
<pre> java.io.FileOutputStream FileOutputStream(FileDescriptor fdObj) </pre>	checkWrite(FileDescriptor)	java.lang.RuntimePermission "writeFileDescriptor"
<pre> java.io.FileOutputStream FileOutputStream(File file) FileOutputStream(String name) FileOutputStream(String name, boolean append) java.io.File public boolean canWrite() public boolean createNewFile() public static File createTempFile(String prefix, String suffix) public static File createTempFile(String prefix, String suffix, File directory) public boolean mkdir() public boolean mkdirs() public boolean renameTo(File dest) public boolean setLastModified(long time) public boolean setReadOnly() </pre>	checkWrite(String)	java.io.FilePermission "{name}", "write"
<pre> java.io.ObjectInputStream protected final boolean enableResolveObject(boolean enable); java.io.ObjectOutputStream protected final boolean enableReplaceObject(boolean enable) </pre>	checkPermission	java.io.SerializablePermission "enableSubstitution"
<pre> java.io.ObjectInputStream protected ObjectInputStream() java.io.ObjectOutputStream protected ObjectOutputStream() </pre>	checkPermission	java.io.SerializablePermission "enableSubclassImplementation"

<pre>java.io.RandomAccessFile RandomAccessFile(String name, String mode) (where mode is "rw")</pre>	<pre>checkRead(String) and checkWrite(String)</pre>	<pre>java.io.FilePermission "{name}", "read,write"</pre>
<pre>java.lang.Class public static Class forName(String name, boolean initialize, ClassLoader loader)</pre>	<pre>checkPermission</pre>	If <code>loader</code> is null, and the caller's class loader is not null, then <pre>java.lang.RuntimePermission("getClassLoader")</pre>
<pre>java.lang.Class public ClassLoader getClassLoader()</pre>	<pre>checkPermission</pre>	If the caller's class loader is null, or is the same as or an ancestor of the class loader for the class whose class loader is being requested, no permission is needed. Otherwise, <pre>java.lang.RuntimePermission "getClassLoader"</pre> is required.
<pre>java.lang.Class public Class[] getDeclaredClasses() public Field[] getDeclaredFields() public Method[] getDeclaredMethods() public Constructor[] getDeclaredConstructors() public Field getDeclaredField(String name) public Method getDeclaredMethod(...) public Constructor getDeclaredConstructor(...)</pre>	<pre>checkMemberAccess(this, Member.DECLARED) and, if this class is in a package, checkPackageAccess({pkgName})</pre>	Default <code>checkMemberAccess</code> does not require any permissions if "this" class's classloader is the same as that of the caller. Otherwise, it requires <code>java.lang.RuntimePermission "accessDeclaredMembers"</code>. If this class is in a package, <code>java.lang.RuntimePermission "accessClassInPackage.{pkgName}"</code> is also required.
<pre>java.lang.Class public Class[] getClasses() public Field[] getFields() public Method[] getMethods() public Constructor[] getConstructors() public Field getField(String name) public Method getMethod(...) public Constructor getConstructor(...)</pre>	<pre>checkMemberAccess(this, Member.PUBLIC) and, if class is in a package, checkPackageAccess({pkgName})</pre>	Default <code>checkMemberAccess</code> does not require any permissions when the access type is <code>Member.PUBLIC</code>. If this class is in a package, <code>java.lang.RuntimePermission "accessClassInPackage.{pkgName}"</code> is required.
<pre>java.lang.Class public ProtectionDomain getProtectionDomain()</pre>	<pre>checkPermission</pre>	<pre>java.lang.RuntimePermission "getProtectionDomain"</pre>

<pre>java.lang.ClassLoader ClassLoader() ClassLoader(ClassLoader parent)</pre>	checkCreateClassLoader	java.lang.RuntimePermission "createClassLoader"
<pre>java.lang.ClassLoader public static ClassLoader getSystemClassLoader() public ClassLoader getParent()</pre>	checkPermission	If the caller's class loader is null, or is the same as or an ancestor of the class loader for the class whose class loader is being requested, no permission is needed. Otherwise, java.lang.RuntimePermission "getClassLoader" is required.
<pre>java.lang.Runtime public Process exec(String command) public Process exec(String command, String envp[]) public Process exec(String cmdarray[], String envp[]) public Process exec(String cmdarray[], String envp[])</pre>	checkExec	java.io.FilePermission "{command}", "execute"
<pre>java.lang.Runtime public void exit(int status) public static void runFinalizersOnExit(boolean value) java.lang.System public static void exit(int status) public static void runFinalizersOnExit(boolean value)</pre>	checkExit(status) where status is 0 for runFinalizersOnExit	java.lang.RuntimePermission "exitVM.{status}"
<pre>java.lang.Runtime public void addShutdownHook(Thread hook) public boolean removeShutdownHook(Thread hook)</pre>	checkPermission	java.lang.RuntimePermission "shutdownHooks"
<pre>java.lang.Runtime public void load(String lib) public void loadLibrary(String lib) java.lang.System public static void load(String filename) public static void loadLibrary(String libname)</pre>	checkLink({libName}) where {libName} is the lib, filename or libname argument	java.lang.RuntimePermission "loadLibrary.{libName}"

java.lang.SecurityManager methods	checkPermission	See the next table .
java.lang.System public static Properties getProperties() public static void setProperties(Properties props)	checkPropertiesAccess	java.util.PropertyPermission "*", "read,write"
java.lang.System public static String getProperty(String key) public static String getProperty(String key, String def)	checkPropertyAccess	java.util.PropertyPermission "{key}", "read"
java.lang.System public static void setIn(InputStream in) public static void setOut(PrintStream out) public static void setErr(PrintStream err)	checkPermission	java.lang.RuntimePermission "setIO"
java.lang.System public static String setProperty(String key, String value)	checkPermission	java.util.PropertyPermission "{key}", "write"
java.lang.System public static synchronized void setSecurityManager(SecurityManager s)	checkPermission	java.lang.RuntimePermission "setSecurityManager"
java.lang.Thread public ClassLoader getContextClassLoader()	checkPermission	If the caller's class loader is null, or is the same as or an ancestor of the context class loader for the thread whose context class loader is being requested, no permission is needed. Otherwise, java.lang.RuntimePermission "getClassLoader" is required.
java.lang.Thread public void setContextClassLoader (ClassLoader cl)	checkPermission	java.lang.RuntimePermission "setContextClassLoader"
java.lang.Thread	checkAccess(this)	java.lang.RuntimePermission "modifyThread"

<pre> public final void checkAccess() public void interrupt() public final void suspend() public final void resume() public final void setPriority (int newPriority) public final void setName(String name) public final void setDaemon(boolean on) </pre>		
<pre> java.lang.Thread public static int enumerate(Thread tarray[]) </pre>	checkAccess({threadGroup})	java.lang.RuntimePermission "modifyThreadGroup"
<pre> java.lang.Thread public final void stop() </pre>	checkAccess(this). Also checkPermission if the current thread is trying to stop a thread other than itself.	java.lang.RuntimePermission "modifyThread". Also java.lang.RuntimePermission "stopThread" if the current thread is trying to stop a thread other than itself.
<pre> java.lang.Thread public final synchronized void stop(Throwable obj) </pre>	checkAccess(this). Also checkPermission if the current thread is trying to stop a thread other than itself or obj is not an instance of ThreadDeath.	java.lang.RuntimePermission "modifyThread". Also java.lang.RuntimePermission "stopThread" if the current thread is trying to stop a thread other than itself or obj is not an instance of ThreadDeath.
<pre> java.lang.Thread Thread() Thread(Runnable target) Thread(String name) Thread(Runnable target, String name) java.lang.ThreadGroup ThreadGroup(String name) ThreadGroup(ThreadGroup parent, String name) </pre>	checkAccess({parentThreadGroup})	java.lang.RuntimePermission "modifyThreadGroup"
<pre> java.lang.Thread Thread(ThreadGroup group, ...) java.lang.ThreadGroup public final void checkAccess() public int enumerate(Thread list[]) public int enumerate(Thread list[], </pre>	checkAccess(this) for ThreadGroup methods, or checkAccess(group) for Thread methods	java.lang.RuntimePermission "modifyThreadGroup"

<pre> boolean recurse) public int enumerate(ThreadGroup list[]) public int enumerate(ThreadGroup list[], boolean recurse) public final ThreadGroup getParent() public final void setDaemon(boolean daemon) public final void setMaxPriority(int pri) public final void suspend() public final void resume() public final void destroy() </pre>		
<pre> java.lang.ThreadGroup public final void interrupt() </pre>	checkAccess(this)	Requires java.lang.RuntimePermission "modifyThreadGroup". Also requires java.lang.RuntimePermission "modifyThread", since the java.lang.Thread interrupt() method is called for each thread in the thread group and in all of its subgroups. See the Thread interrupt() method.
<pre> java.lang.ThreadGroup public final void stop() </pre>	checkAccess(this)	Requires java.lang.RuntimePermission "modifyThreadGroup". Also requires java.lang.RuntimePermission "modifyThread" and possibly java.lang.RuntimePermission "stopThread", since the java.lang.Thread stop() method is called for each thread in the thread group and in all of its subgroups. See the Thread stop() method.
<pre> java.lang.reflect.AccessibleObject public static void setAccessible(...) public void setAccessible(...) </pre>	checkPermission	java.lang.reflect.ReflectPermission "suppressAccessChecks"
<pre> java.net.Authenticator public static PasswordAuthentication requestPasswordAuthentication(InetAddress addr, int port, String protocol, String prompt, String scheme) </pre>	checkPermission	java.net.NetPermission "requestPasswordAuthentication"

<pre>java.net.Authenticator public static void setDefault(Authenticator a)</pre>	checkPermission	java.net.NetPermission "setDefaultAuthenticator"
<pre>java.net.MulticastSocket public void joinGroup(InetAddress mcastaddr) public void leaveGroup(InetAddress mcastaddr)</pre>	checkMulticast(InetAddress)	java.net.SocketPermission(mcastaddr.getHostAddress(), "accept,connect")
<pre>java.net.DatagramSocket public void send(DatagramPacket p)</pre>	checkMulticast(p.getAddress()) or checkConnect(p.getAddress().getHostAddress(), p.getPort())	if (p.getAddress().isMulticastAddress()) { java.net.SocketPermission((p.getAddress()).getHostAddress(), "accept,connect") } else { port = p.getPort(); host = p.getAddress().getHostAddress(); if (port == -1) java.net.SocketPermission " {host}", "resolve"; else java.net.SocketPermission "{host}: {port}", "connect" } }
<pre>java.net.MulticastSocket public synchronized void send(DatagramPacket p, byte ttl)</pre>	checkMulticast(p.getAddress(), ttl) or checkConnect(p.getAddress().getHostAddress(), p.getPort())	if (p.getAddress().isMulticastAddress()) { java.net.SocketPermission((p.getAddress()).getHostAddress(), "accept,connect") } else { port = p.getPort(); host = p.getAddress().getHostAddress(); if (port == -1) java.net.SocketPermission " {host}", "resolve"; else java.net.SocketPermission "{host}: {port}", "connect" } }
<pre>java.net.InetAddress public String getHostName() public static InetAddress[] getAllByName(String host) public static InetAddress getLocalHost()</pre>	checkConnect({host}, -1)	java.net.SocketPermission "{host}", "resolve"

<pre>java.net.DatagramSocket public InetAddress getLocalAddress()</pre>		
<pre>java.net.ServerSocket ServerSocket(...) java.net.DatagramSocket DatagramSocket(...) java.net.MulticastSocket MulticastSocket(...)</pre>	<pre>checkListen({port})</pre>	<pre>java.net.SocketPermission "localhost: {port}", "listen";</pre>
<pre>java.net.ServerSocket public Socket accept() protected final void implAccept(Socket s)</pre>	<pre>checkAccept({host}, {port})</pre>	<pre>java.net.SocketPermission "{host}:{port}", "accept"</pre>
<pre>java.net.ServerSocket public static synchronized void setSocketFactory(...) java.net.Socket public static synchronized void setSocketImplFactory(...) java.net.URL public static synchronized void setURLStreamHandlerFactory(...) java.net.URLConnection public static synchronized void setContentHandlerFactory(...) public static void setFileNameMap(FileNameMap map) java.net.HttpURLConnection public static void setFollowRedirects(boolean set) java.rmi.activation.ActivationGroup public static synchronized</pre>	<pre>checkSetFactory</pre>	<pre>java.lang.RuntimePermission "setFactory"</pre>

<pre> ActivationGroup createGroup(...) public static synchronized void setSystem(ActivationSystem system) java.rmi.server.RMISocketFactory public synchronized static void setSocketFactory(...) </pre>		
<pre> java.net.Socket Socket(...) </pre>	checkConnect({host}, {port})	java.net.SocketPermission "{host}:{port}", "connect"
<pre> java.net.DatagramSocket public synchronized void receive(DatagramPacket p) </pre>	checkAccept({host}, {port})	java.net.SocketPermission "{host}:{port}", "accept"
<pre> java.net.URL URL(...) </pre>	checkPermission	java.net.NetPermission "specifyStreamHandler"
<pre> java.net.URLClassLoader URLClassLoader(...) </pre>	checkCreateClassLoader	java.lang.RuntimePermission "createClassLoader"
<pre> java.security.AccessControlContext public AccessControlContext(AccessControlContext acc, DomainCombiner combiner) public DomainCombiner getDomainCombiner() </pre>	checkPermission	java.security.SecurityPermission "createAccessControlContext"
<pre> java.security.Identity public void addCertificate(...) </pre>	checkSecurityAccess("addIdentityCertificate")	java.security.SecurityPermission "addIdentityCertificate"
<pre> java.security.Identity public void removeCertificate(...) </pre>	checkSecurityAccess("removeIdentityCertificate")	java.security.SecurityPermission "removeIdentityCertificate"
<pre> java.security.Identity public void setInfo(String info) </pre>	checkSecurityAccess("setIdentityInfo")	java.security.SecurityPermission "setIdentityInfo"
<pre> java.security.Identity public void setPublicKey(PublicKey key) </pre>	checkSecurityAccess("setIdentityPublicKey")	java.security.SecurityPermission "setIdentityPublicKey"

<pre>java.security.Identity public String toString(...)</pre>	<pre>checkSecurityAccess("printIdentity")</pre>	<pre>java.security.SecurityPermission "printIdentity"</pre>
<pre>java.security.IdentityScope protected static void setSystemScope()</pre>	<pre>checkSecurityAccess("setSystemScope")</pre>	<pre>java.security.SecurityPermission "setSystemScope"</pre>
<pre>java.security.Permission public void checkGuard(Object object)</pre>	<pre>checkPermission(this)</pre>	this Permission object is the permission checked
<pre>java.security.Policy public static Policy getPolicy()</pre>	<pre>checkPermission</pre>	<pre>java.security.SecurityPermission "getPolicy"</pre>
<pre>java.security.Policy public static void setPolicy(Policy policy)</pre>	<pre>checkPermission</pre>	<pre>java.security.SecurityPermission "setPolicy"</pre>
<pre>java.security.Policy public static Policy getInstance(String type, SpiParameter params) getInstance(String type, SpiParameter params, String provider) getInstance(String type, SpiParameter params, Provider provider)</pre>	<pre>checkPermission</pre>	<pre>java.security.SecurityPermission "createPolicy. {type}"</pre>
<pre>java.security.Provider public synchronized void clear()</pre>	<pre>checkSecurityAccess("clearProviderProperties."+{name})</pre>	<pre>java.security.SecurityPermission "clearProviderProperties.{name}" where <i>name</i> is the provider name.</pre>
<pre>java.security.Provider public synchronized Object put(Object key, Object value)</pre>	<pre>checkSecurityAccess("putProviderProperty."+{name})</pre>	<pre>java.security.SecurityPermission "putProviderProperty.{name}" where <i>name</i> is the provider name.</pre>
<pre>java.security.Provider public synchronized Object remove(Object key)</pre>	<pre>checkSecurityAccess("removeProviderProperty."+{name})</pre>	<pre>java.security.SecurityPermission "removeProviderProperty.{name}" where <i>name</i> is the provider name.</pre>
<pre>java.security.SecureClassLoader SecureClassLoader(...)</pre>	<pre>checkCreateClassLoader</pre>	<pre>java.lang.RuntimePermission "createClassLoader"</pre>

<pre>java.security.Security public static void getProperty(String key)</pre>	checkPermission	java.security.SecurityPermission "getProperty.{key}"
<pre>java.security.Security public static int addProvider(Provider provider) public static int insertProviderAt(Provider provider, int position);</pre>	checkSecurityAccess("insertProvider."+provider.getName())	java.security.SecurityPermission "insertProvider.{name}"
<pre>java.security.Security public static void removeProvider(String name)</pre>	checkSecurityAccess("removeProvider."+name)	java.security.SecurityPermission "removeProvider.{name}"
<pre>java.security.Security public static void setProperty(String key, String datum)</pre>	checkSecurityAccess("setProperty."+key)	java.security.SecurityPermission "setProperty.{key}"
<pre>java.security.Signer public PrivateKey getPrivateKey()</pre>	checkSecurityAccess("getSignerPrivateKey")	java.security.SecurityPermission "getSignerPrivateKey"
<pre>java.security.Signer public final void setKeyPair(KeyPair pair)</pre>	checkSecurityAccess("setSignerKeypair")	java.security.SecurityPermission "setSignerKeypair"
<pre>java.sql.DriverManager public static synchronized void setLogWriter(PrintWriter out)</pre>	checkPermission	java.sql.SQLPermission "setLog"
<pre>java.sql.DriverManager public static synchronized void setLogStream(PrintWriter out)</pre>	checkPermission	java.sql.SQLPermission "setLog"
<pre>java.util.Locale public static synchronized void setDefault(Locale newLocale)</pre>	checkPermission	java.util.PropertyPermission "user.language","write"

<pre>java.util.zip.ZipFile ZipFile(String name)</pre>	checkRead	java.io.FilePermission "{name}","read"
<pre>javax.security.auth.Subject public static Subject getSubject(final AccessControlContext acc)</pre>	checkPermission	javax.security.auth.AuthPermission "getSubject"
<pre>javax.security.auth.Subject public void setReadOnly()</pre>	checkPermission	javax.security.auth.AuthPermission "setReadOnly"
<pre>javax.security.auth.Subject public static Object doAs(final Subject subject, final PrivilegedAction action)</pre>	checkPermission	javax.security.auth.AuthPermission "doAs"
<pre>javax.security.auth.Subject public static Object doAs(final Subject subject, final PrivilegedExceptionAction action) throws java.security.PrivilegedActionException</pre>	checkPermission	javax.security.auth.AuthPermission "doAs"
<pre>javax.security.auth.Subject public static Object doAsPrivileged(final Subject subject, final PrivilegedAction action, final AccessControlContext acc)</pre>	checkPermission	javax.security.auth.AuthPermission "doAsPrivileged"
<pre>javax.security.auth.Subject public static Object doAsPrivileged(final Subject subject, final PrivilegedExceptionAction action, final AccessControlContext acc) throws java.security.PrivilegedActionException</pre>	checkPermission	javax.security.auth.AuthPermission "doAsPrivileged"
<pre>javax.security.auth.SubjectDomainCombiner public Subject getSubject()</pre>	checkPermission	javax.security.auth.AuthPermission "getSubjectFromDomainCombiner"
<pre>javax.security.auth.SubjectDomainCombiner public Subject getSubject()</pre>	checkPermission	javax.security.auth.AuthPermission "getSubjectFromDomainCombiner"
<pre>javax.security.auth.login.LoginContext public LoginContext(String name)</pre>	checkPermission	javax.security.auth.AuthPermission "createLoginContext.{name}"

throws LoginException		
<pre> javax.security.auth.login.LoginContext public LoginContext(String name, Subject subject) throws LoginException </pre>	checkPermission	<pre> javax.security.auth.AuthPermission "createLoginContext.{name}" </pre>
<pre> javax.security.auth.login.LoginContext public LoginContext(String name, CallbackHandler callbackHandler) throws LoginException </pre>	checkPermission	<pre> javax.security.auth.AuthPermission "createLoginContext.{name}" </pre>
<pre> javax.security.auth.login.LoginContext public LoginContext(String name, Subject subject, CallbackHandler callbackHandler) throws LoginException </pre>	checkPermission	<pre> javax.security.auth.AuthPermission "createLoginContext.{name}" </pre>
<pre> javax.security.auth.login.Configuration public static Configuration getConfiguration() </pre>	checkPermission	<pre> javax.security.auth.AuthPermission "getLoginConfiguration" </pre>
<pre> javax.security.auth.login.Configuration public static void setConfiguration(Configuration configuration) </pre>	checkPermission	<pre> javax.security.auth.AuthPermission "setLoginConfiguration" </pre>
<pre> javax.security.auth.login.Configuration public static void refresh() </pre>	checkPermission	<pre> javax.security.auth.AuthPermission "refreshLoginConfiguration" </pre>
<pre> javax.security.auth.login.Configuration public static Configuration getInstance(String type, SpiParameter params) getInstance(String type, SpiParameter params, String provider) getInstance(String type, SpiParameter params, Provider provider) </pre>	checkPermission	<pre> javax.security.auth.AuthPermission "createLoginConfiguration.{type}" </pre>

java.lang.SecurityManager Method Permission Checks

This table shows which permissions are checked for by the default implementations of the `java.lang.SecurityManager` methods.

Each of the specified `check` methods calls the `SecurityManager checkPermission` method with the specified permission, except for the `checkConnect` and `checkRead` methods that take a context argument. Those methods expect the context to be an `AccessControlContext` and they call the context's `checkPermission` method with the specified permission.

Method	Permission
<code>public void checkAccept(String host, int port);</code>	<code>java.net.SocketPermission "{host}:{port}", "accept";</code>
<code>public void checkAccess(Thread t);</code>	<code>java.lang.RuntimePermission "modifyThread";</code>
<code>public void checkAccess(ThreadGroup g);</code>	<code>java.lang.RuntimePermission "modifyThreadGroup";</code>
<code>public void checkAwtEventQueueAccess();</code> Note: This method is deprecated; use instead <code>public void checkPermission(Permission perm);</code>	<code>java.awt.AWTPermission "accessEventQueue";</code>
<code>public void checkConnect(String host, int port);</code>	if (port == -1) <code>java.net.SocketPermission "{host}", "resolve";</code> else <code>java.net.SocketPermission "{host}:{port}", "connect";</code>
<code>public void checkConnect(String host, int port, Object context);</code>	if (port == -1) <code>java.net.SocketPermission "{host}", "resolve";</code> else <code>java.net.SocketPermission "{host}:{port}", "connect";</code>
<code>public void checkCreateClassLoader();</code>	<code>java.lang.RuntimePermission "createClassLoader";</code>
<code>public void checkDelete(String file);</code>	<code>java.io.FilePermission "{file}", "delete";</code>
<code>public void checkExec(String cmd);</code>	if cmd is an absolute path: <code>java.io.FilePermission "{cmd}", "execute";</code> else <code>java.io.FilePermission "<<ALL_FILES>>", "execute";</code>
<code>public void checkExit(int status);</code>	<code>java.lang.RuntimePermission "exitVM.{status}";</code>
<code>public void checkLink(String lib);</code>	<code>java.lang.RuntimePermission "loadLibrary.{lib}";</code>
<code>public void checkListen(int port);</code>	<code>java.net.SocketPermission "localhost:{port}", "listen";</code>
<code>public void checkMemberAccess(Class clazz, int which);</code> Note: This method is deprecated; use instead <code>public void checkPermission(Permission perm);</code>	<pre> if (which != Member.PUBLIC) { if (currentClassLoader() != clazz.getClassLoader()) { checkPermission(new java.lang.RuntimePermission("accessDeclaredMembers")); } } </pre>

public void checkMulticast(InetAddress maddr);	java.net.SocketPermission(maddr.getHostAddress(),"accept,connect");
public void checkMulticast(InetAddress maddr, byte ttl); Note: This method is deprecated; use instead public void checkPermission(Permission perm);	java.net.SocketPermission(maddr.getHostAddress(),"accept,connect");
public void checkPackageAccess(String pkg);	java.lang.RuntimePermission "accessClassInPackage.{pkg}";
public void checkPackageDefinition(String pkg);	java.lang.RuntimePermission "defineClassInPackage.{pkg}";
public void checkPrintJobAccess();	java.lang.RuntimePermission "queuePrintJob";
public void checkPropertiesAccess();	java.util.PropertyPermission "*", "read,write";
public void checkPropertyAccess(String key);	java.util.PropertyPermission "{key}", "read,write";
public void checkRead(FileDescriptor fd);	java.lang.RuntimePermission "readFileDescriptor";
public void checkRead(String file);	java.io.FilePermission "{file}", "read";
public void checkRead(String file, Object context);	java.io.FilePermission "{file}", "read";
public void checkSecurityAccess(String target);	java.security.SecurityPermission "{target}";
public void checkSetFactory();	java.lang.RuntimePermission "setFactory";
public void checkSystemClipboardAccess(); Note: This method is deprecated; use instead public void checkPermission(Permission perm);	java.awt.AWTPermission "accessClipboard";
public boolean checkTopLevelWindow(Object window); Note: This method is deprecated; use instead public void checkPermission(Permission perm);	java.awt.AWTPermission "showWindowWithoutWarningBanner";
public void checkWrite(FileDescriptor fd);	java.lang.RuntimePermission "writeFileDescriptor";
public void checkWrite(String file);	java.io.FilePermission "{file}", "write";
public SecurityManager();	java.lang.RuntimePermission "createSecurityManager";